



FULL-STACK APPLICATION DESIGN AND DEVELOPMENT

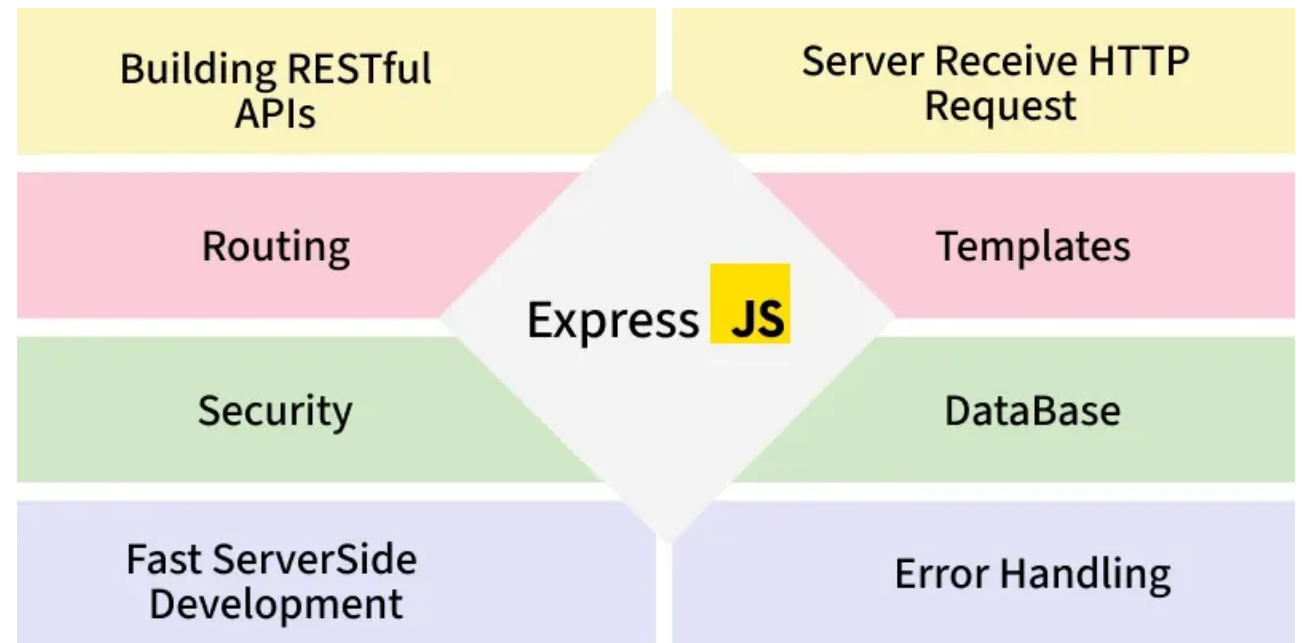
COURSE 7 - EXPRESS



INTRODUCTION

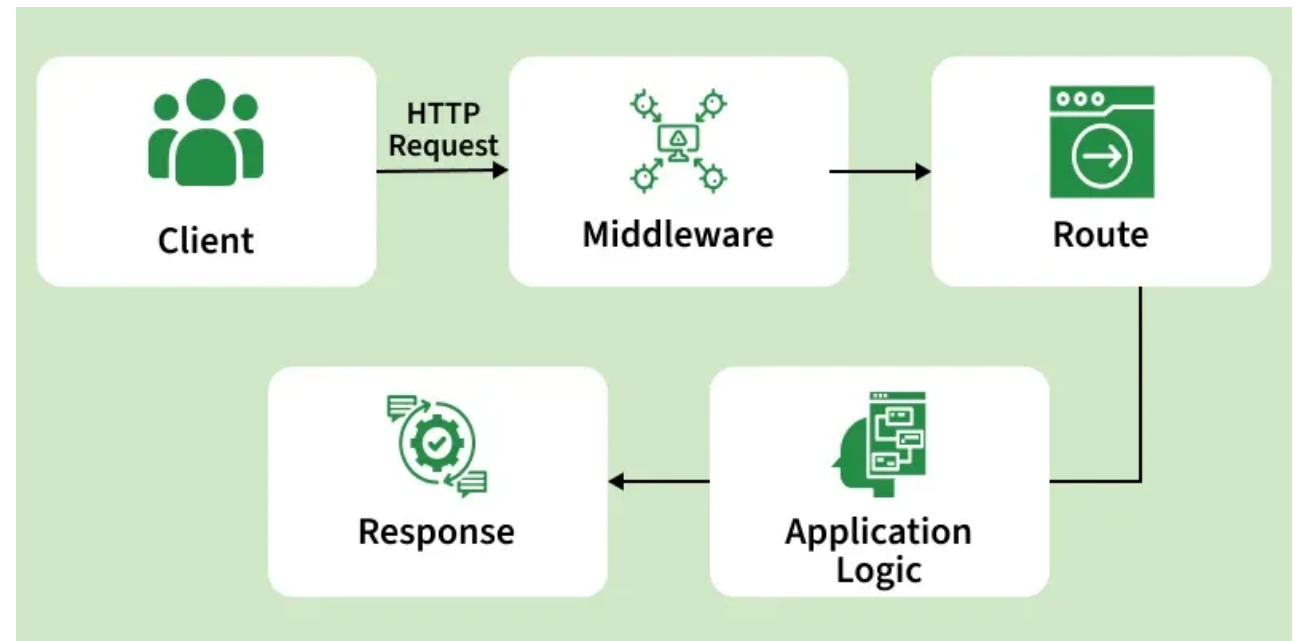
Express.js is a minimal and flexible Node.js web application framework that simplifies building server-side applications and APIs.

- **Middleware-based:** Uses middleware functions to handle requests, responses, and add features like authentication, logging, or error handling.
- **Routing system:** structure - developers can design apps as they want.
- **Integration:** Works well with databases (MongoDB, MySQL, PostgreSQL) and front-end frameworks.
- **Scalable:** Suitable for small apps to large-scale enterprise APIs.



INTRODUCTION

- Express.js works by handling client requests through a series of steps called the **request-response cycle**, using middleware and routing to process requests and send responses efficiently.
 - A client sends a request (e.g., browser or app) to the Express server.
 - The request passes through middleware functions that can modify the request, perform logging, authentication, or other tasks.
 - Express matches the request to a route handler based on the URL and HTTP method.
 - The route handler processes the request and prepares a response.
 - The server sends the response back to the client.
 - If any error occurs, it is caught by error-handling middleware to ensure the server remains stable.



INTRODUCTION

- Basic routing in Express.js is the process of defining URL endpoints (routes) and specifying how the server should respond to client requests at those endpoints.
- Express provides simple methods to define routes that correspond to HTTP methods:
 - `app.get()` - Handle GET requests
 - `app.post()` - Handle POST requests
 - `app.put()` - Handle PUT requests
 - `app.delete()` - Handle DELETE requests
 - `app.all()` - Handle all HTTP methods

INTRODUCTION

- Parameters in Express.js are values that are passed in the URL or request that your server can use to process the request dynamically. They allow your routes to handle different inputs without creating multiple static routes.

- **Types of Parameters in Express:**

1. **Route Parameters:** Part of the URL (e.g., /users/:id) accessed via req.params.

```
app.get('/users/:id', (req, res) => {  
  const userId = req.params.id;  
  res.send(`User ID is ${userId}`);  
});
```

2. **Query Parameters:** Appended to the URL (e.g., /search?term=nodejs) accessed via req.query.

```
app.get('/search', (req, res) => {  
  const searchTerm = req.query.term;  
  res.send(`Searching for ${searchTerm}`);  
});
```

INTRODUCTION

3. **Request Body Parameters:** Sent in POST or PUT requests, accessed via req.body with express.json() middleware.

```
app.use(express.json());
app.post('/users', (req, res) => {
  const { name, age } = req.body;
  res.send(`User created: ${name}, Age: ${age}`);
});
```

INTRODUCTION

- Middleware in Express.js is a function that executes during the request-response cycle. It has access to the request (req) and response (res) objects, and can either end the request-response process or pass control to the next middleware using next().
- **Middleware is used for:**
 - Logging requests
 - Authentication and authorization
 - Parsing request bodies
 - Handling errors

INTRODUCTION

- Express includes several useful middleware functions:
 - `express.json()` - Parse JSON request bodies
 - `express.urlencoded()` - Parse URL-encoded request bodies
 - `express.static()` - Serve static files
 - `express.Router()` - Create modular route handlers

```
const express = require('express');
const app = express();
const PORT = 3000;

// Middleware to log requests
app.use((req, res, next) => {
  console.log(`${req.method} request for ${req.url}`);
  next(); // Pass control to the next middleware/route
});

// Middleware to parse JSON bodies
app.use(express.json());

// Route handler
app.get('/', (req, res) => {
  res.send('Hello, Express with middleware!');
});

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```


INTRODUCTION

- Error handling in Express is done through special middleware functions that have four arguments: `err`, `req`, `res`, `next`.

```
const express = require('express');
const app = express();
const PORT = 3000;

// Middleware to simulate an error
app.get('/error', (req, res, next) => {
  const err = new Error('Something went wrong!');
  err.status = 500;
  next(err); // Pass error to error-handling
  middleware
});
```

```
// Error-handling middleware
app.use((err, req, res, next) => {
  console.error(err.stack); // Log the error
  res.status(err.status || 500).send({
    message: err.message,
    status: err.status || 500
  });
});

app.listen(PORT, () => {
  console.log(`Server running on
http://localhost:${PORT}`);
});
```

INTRODUCTION

- Express.js is a popular web framework for Node.js that simplifies server-side development. It is widely used to build web applications, APIs, and scalable server-side solutions.
 - **Web Applications:** Express.js is ideal for building dynamic websites and single-page applications (SPAs) that require fast and efficient server-side handling.
 - **RESTful APIs:** It's widely used to create APIs that handle CRUD operations, serving data to front-end applications or mobile apps.
 - **Middleware-based Applications:** Express allows integration of middleware for logging, authentication, input validation, and other request processing tasks.
 - **Microservices:** Its lightweight and modular design makes it suitable for developing microservices in a scalable architecture.
 - **Server-side Rendering:** Works seamlessly with template engines like EJS, Pug, or Handlebars to render HTML pages on the server.
 - **Proxy and Gateway Servers:** Can act as a proxy or gateway, routing requests to different services and managing HTTP traffic efficiently.
 - **Rapid Prototyping:** Developers often use Express for quickly setting up server-side logic and testing ideas before scaling applications.

CREATE AN EXPRESS.JS APPLICATION IN LINUX

1. Install Node.js on your system.

2. Create a Project Directory

- Open your terminal/command prompt and create a new folder for your project:

```
mkdir my-express-app
```

```
cd my-express-app
```

3. Initialize npm

- Initialize a new Node.js project using npm:

```
npm init or npm init -y
```

4. Install Express

- Install Express.js as a dependency:

```
npm install express
```

CREATE AN EXPRESS.JS APPLICATION IN LINUX

5. Create the Server File

- Create an `app.js` (or `server.js`) file. This will serve as the main entry point for your application.
- **Project Structure:**

```
> node_modules
  app.js
  package-lock.json
  package.json
```

CREATE AN EXPRESS.JS APPLICATION IN LINUX

Approach:

- Use `require('express')` to import the Express module.
- Call `express()` to create an Express application instance.
- Define the port for the application, typically 3000.
- Set up a basic GET route with `app.get('/', (req, res) => res.send('Hello World!'))`.
- Use `app.listen` method to listen to your desired PORT and to start the server.

CREATE AN EXPRESS.JS APPLICATION IN LINUX

- **Example:** Implementation to setup the express application.

```
const express = require('express');
const app = express();
const PORT = 3000;

app.listen(PORT, (error) =>{
  if(!error)
    console.log("Server is Successfully Running,
                and App is listening on port "+ PORT);
  else
    console.log("Error occurred, server can't start", error);
});
```

- Run the application using the following command from the root directory of the project:

```
node app.js
```

CREATE AN EXPRESS.JS APPLICATION IN LINUX

6. Setting Up Routes

- Routes are server endpoints that respond to client requests. You can think of a route as a function that runs when someone accesses a specific path. Express supports routes for various HTTP methods like GET, POST, PUT, and more.

`app.anyMethod(path, function)`

- **Setting GET request route on the root URL:**

- Use `app.get()` to configure the route with the path `'/'` and a callback function.
- The callback function receives `req` (request) and `res` (response) objects provided by Express.
- `req` is the incoming request object containing client data, and `res` is the response object used to send data back to the client.
- Use `res.status()` to set the HTTP status code before sending the response.
- Use `res.send()` to send the response back to the client. You can send a string, object, array, or buffer. Other response methods include `res.json()` for JSON objects and `res.sendFile()` files.

CREATE AN EXPRESS.JS APPLICATION IN LINUX

- **Example:** Implementation to set up a basic request route on the root URL.

<pre>//app.js const express = require('express'); const app = express(); const PORT = 3000; app.get('/', (req, res)=>{ res.status(200); res.send("Welcome to root URL of Server"); });</pre>	<pre>app.listen(PORT, (error) =>{ if(!error) console.log("Server is Successfully Running, and App is listening on port "+ PORT); else console.log("Error occurred, server can't start", error); } });</pre>
---	---

- Save the code, restart the server, and open localhost:<port>. A GET request to / will return the plain text response, which you can also inspect in Chrome DevTools under the Network tab.

CREATE AN EXPRESS.JS APPLICATION IN LINUX

7. Sending Data to the Server

- Sometimes we need to send data to the server (e.g., login details). This is done using the request object with methods like POST. Since browsers can only send GET requests via the address bar, tools like Postman or a frontend form are needed to send data.
- **Setting route to be accessed by users to send data with post requests.**
 - Use `express.json()` middleware to parse incoming JSON data.
 - Apply middleware with `app.use()`.
 - Create a POST route on `'/'`.
 - Extract the name property from `req.body` using ES6 syntax: `const { name } = req.body`.
 - Send a response confirming the data was received.
 - Use `${}` to include variables in the response string.

CREATE AN EXPRESS.JS APPLICATION IN LINUX

- **Example:** Implementation to Setup route to be accessed by users to send data with post requests.

<pre>// app.js const express = require('express'); const app = express(); const PORT = 3000; app.use(express.json()); app.post('/', (req, res)=>{ const {name} = req.body; res.send(`Welcome \${name}`); })</pre>	<pre>app.listen(PORT, (error) =>{ if(!error) console.log("Server is Successfully Running, and App is listening on port "+ PORT); else console.log("Error occurred, server can't start", error); } });</pre>
---	---

- Test the route using Postman (or any tool like Axios, fetch, or cURL). Sending JSON data in the request body returns a response from the server, confirming the data was received successfully.

CREATE AN EXPRESS.JS APPLICATION IN LINUX

8. Sending Files from Server

- To send files from the server, we can either serve static files using middleware or send a single file via a route.
- **Example 1: Serving entire directory using middleware**
- In Express, we use the middleware `express.static()` function, which takes two arguments. The first argument is the absolute root path of the directory containing the files we want to serve. We can easily serve static files by using `app.use()` and providing the `express.static()` middleware.

```
app.use(path, express.static(root, [options]));
```

CREATE AN EXPRESS.JS APPLICATION IN LINUX

- **Example:** Implementation to show the use of middleware in Express.

```
// app.js

const express = require('express');
const app = express();
const PORT = 3000;

const path = require('path')
app.use('/static',
express.static(path.join(__dirname, 'Static
Files')))
```

```
app.listen(PORT, (error) =>{
    if(!error)
        console.log("Server is Successfully
Running,                and App is listening on port
"+ PORT);
    else
        console.log("Error occurred, server
can't start", error);
    }
});
```

- This will be the returned response when we request some static file from the directory that we are serving as static. Here you can see we have received an HTML file as a response for **'/static/random.html'**. The same things happen when we request for **'/static/1.jpg'**.

CREATE AN EXPRESS.JS APPLICATION IN LINUX

- **Example 2: Sending a single file on a route with the `sendFile()` function**
- This function accepts an absolute URL of the file and whenever the route path is being accessed the server provides the file as an HTTP response. This process can be thought of as a single endpoint of the `express.static()`. It can be useful when we have to do some kind of processing before sending the file.

```
res.sendFile(fileUrl)
```

CREATE AN EXPRESS.JS APPLICATION IN LINUX

- **Example:** Implementation to show Sending a single file on a route with the `sendFile()` function.

<pre>// app.js const express = require('express'); const path = require('path'); const app = express(); const PORT = 3000; app.get('/file', (req, res)=>{ res.sendFile(path.join(__dirname, 'image.jpg')); });</pre>	<pre>app.listen(PORT, (error) =>{ if(!error) console.log("Server is Successfully Running, and App is listening on port "+ PORT); else console.log("Error occurred, server can't start", error); } });</pre>
---	--

- **Output:** After running the server, When we request the route **'/file'** the server sends the `image.jpg` file as a response.

UNIQUE FEATURES OF EXPRESS.JS

- **Minimalist Framework** - Express.js is lightweight and focuses on only the essential features needed to create a web server or API.
- **Powerful Routing System** - Express.js provides a powerful routing system that allows you to define routes for different HTTP methods (GET, POST, PUT, DELETE) and URLs. This makes it flexible for handling various requests.
- **Middleware Support** - Express.js allows you to use middleware functions to process requests and responses. Middleware can be applied globally or to specific routes, helping organize your application logic.
- **Templating Engines** - Express allows you to use various templating engines like Pug, EJS, or Handlebars.
- **Asynchronous Request Handling** - Since Express is built on top of Node.js, it supports asynchronous request handling. This allows the server to handle multiple requests at once without blocking, improving performance.

SENDING A RESPONSE FROM SERVER TO CLIENT

- In Express.js, serving static files like images, CSS, and JavaScript used to require custom routes. With the `express.static()` function, you can serve static content directly from a folder, making it easier and faster.
- **What is `express.static()`?**
- The `express.static()` is a built-in middleware function in Express.js that allows you to serve static files (like images, HTML, CSS, and JavaScript) directly to the client. It automatically makes all files inside a specified folder accessible via HTTP requests. You don't have to create custom routes for each file.

```
app.use(express.static('directory_name'));
```


SENDING A RESPONSE FROM SERVER TO CLIENT

- **Code implementation of `express.static()`**
- **Step 1: Create a public Folder**
- First, create a folder named `public` in your project directory. Inside this folder, place an `index.html` file and any images you want to display.
- **Step 2: Make a `server.js` file**
- Make a `server.js` file that contains the code logic to send the image to the specified route on to your localhost route.

SENDING A RESPONSE FROM SERVER TO CLIENT

```
//server.js
const express = require('express');
const app = express();
// Use express.static middleware to serve static files from the "public" folder
app.use(express.static('public'));
// Define a route (this is optional)
app.get('/', (req, res) => {
  res.send('Welcome to my static website!');
});
// Start the server
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

SENDING A RESPONSE FROM SERVER TO CLIENT

- **Middleware Setup:** `express.static` is a built-in middleware function in Express that serves static files.
- **Directory Path:** It takes a directory path as an argument, indicating where static assets (like images, CSS, and JavaScript) are stored.
- **Request Handling:** When a request is made for a specific file, Express searches the specified directory for the file.
- **File Delivery:** If the requested file exists, Express sends it directly to the client (browser) with the appropriate content type.
- **404 Error:** If the file is not found, Express returns a 404 error, indicating the resource is not available.

SENDING A RESPONSE FROM SERVER TO CLIENT

- **Step 3: Run the server**
- Use the following command to run the server.js file in your current working directory of the project
`node server.js`
- **Step 4: Open Your Browser**
- Visit the localhost:3000 to see the image that has been rendered on the page

SENDING A RESPONSE FROM SERVER TO CLIENT

■ How the `express.static()` works?

- `express.static` is a built-in middleware function in `Express` that serves static files.
- It takes a directory path as an argument, indicating where static assets (like images, CSS, and JavaScript) are stored.
- When a request is made for a specific file, Express searches the specified directory for the file.
- If the requested file exists, Express sends it directly to the client (browser) with the appropriate content type.
- If the file is not found, Express returns a 404 error, indicating the resource is not available.

EXPRESS 'APP' AND 'SERVER' FILES

- The app.js file contains the core logic of your Express application. It's responsible for defining routes, middleware, and how requests should be handled.
 - **Setting up Express:** The app.js file creates and configures the Express application
 - **Handling Middleware:** Middleware like `express.json()` is used for parsing incoming requests.
 - **Defining Routes:** You define the routes that handle HTTP requests in this file.
 - **Exporting the App:** The Express app is exported so it can be used in another file (like `server.js`) to run the application.

EXPRESS 'APP' AND 'SERVER' FILES

```
//app.js
const express = require('express');
const app = express();
// Middleware to parse JSON bodies
app.use(express.json());
// Sample route
app.get('/', (req, res) => {
  res.send('Hello, World!');
});
// Another route with parameters
app.get('/user/:id', (req, res) => {
  const userId = req.params.id;
  res.send(`User ID is ${userId}`);
});
// Export the app to be used by the server
module.exports = app;
```

EXPRESS 'APP' AND 'SERVER' FILES

- The server.js file is responsible for configuring and starting the server. It imports the Express app from app.js, sets the port, and tells the app to listen for incoming requests.
 - **Importing the Express App:** It imports the app created in app.js.
 - **Setting Up the Port:** The server listens on a specific port (e.g., 3000).
 - **Starting the Server:** The server is started using app.listen() to handle incoming requests.
 - **Logging the Server Status:** It logs a message when the server starts successfully.

EXPRESS 'APP' AND 'SERVER' FILES

```
//server.js
const app = require('./app'); // Import the Express app from app.js

const PORT = process.env.PORT || 3000; // Set the port to use (default is 3000)

app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

ERROR HANDLING IN EXPRESS

- Error handling in Express refers to the process of managing errors that occur during the execution of a request. When an error occurs in a route or middleware, Express provides mechanisms to catch these errors and prevent the application from crashing. Error-handling middleware allows developers to:
 - Log errors for debugging.
 - Send structured error responses to users.
 - Maintain application stability even when issues arise.

ERROR HANDLING IN EXPRESS

- Express ensures errors are caught and managed effectively to prevent application crashes.
 - **Error Detection:** Automatically detects issues, such as bad requests or server failures.
 - **Error-Handling Middleware:** Uses special middleware to catch and process errors in a structured way.
 - **Passing Errors:** Errors in routes are passed to the middleware using `next(err)`.
 - **Logging:** Errors are logged to help developers quickly identify and resolve issues.
 - **User-Friendly Responses:** Express sends clear, non-technical messages to users instead of showing stack traces.

ERROR HANDLING IN EXPRESS

I. Default Error Handler

- Express has a built-in error handler that catches any errors that are not handled. If you don't create your own error handler, Express will automatically send a simple "500 Internal Server Error" response.

```
const express = require("express");
const app = express();
app.get("/", (req, res) => {
  throw new Error("Something went wrong!");
});
app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

ERROR HANDLING IN EXPRESS

2. Custom Error-Handling Middleware

- Instead of relying on the built-in error handler, you can create a custom error middleware. This lets you log errors and send user-friendly responses.

```
const express = require("express");
const app = express();
app.get("/", (req, res) => {
  throw new Error("Something broke!");
});
// Custom error-handling middleware
app.use((err, req, res, next) => {
  console.log(err);
  res.status(500).json({ message: "Oops! Something went wrong." });
});
app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

ERROR HANDLING IN EXPRESS

3. Synchronous Error Handling

- Express automatically catches errors in synchronous functions. If an error is thrown inside a route, it is sent to the error handler.

```
const express = require("express");
const app = express();
app.get("/sync-error", (req, res) => {
  throw new Error("Synchronous error occurred!");
});
// Custom error handler
app.use((err, req, res, next) => {
  res.status(500).json({ message: err.message });
});
app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

ERROR HANDLING IN EXPRESS

4. Asynchronous Error Handling

- Unlike errors in normal functions, errors in async functions are not caught automatically by Express. You need to handle them manually using try-catch or pass them to Express using next(err).

```
const express = require("express");
const app = express();
app.get("/async-error", async (req, res, next) => {
  try {
    await Promise.reject(new Error("Async error occurred!"));
  } catch (err) {
    next(err);
  }
});
// Error handler middleware
app.use((err, req, res, next) => {
  res.status(500).json({ message: err.message });
});
app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

ERROR HANDLING IN EXPRESS

5. Handling Errors with next(err)

- Calling next(err) inside a route manually forwards errors to Express's error handler.

```
const express = require("express");
const app = express();
app.get("/manual-error", (req, res, next) => {
  const err = new Error("Manually triggered error!");
  next(err);
});
app.use((err, req, res, next) => {
  res.status(500).json({ message: err.message });
});
app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```


EXPRESS APPLICATION FUNCTIONS

- The **app.all()** function is used to route all types of HTTP requests. Like if we have POST, GET, PUT, DELETE, etc, requests made to any specific route, let's say */user*, so instead of defining different APIs like `app.post('/user')`, `app.get('/user')`, etc, we can define single API **app.all('/user')** which will accept all type of HTTP request.
- **Syntax:**
`app.all( path, callback )`
- **Parameters:**
 - **Path:** It is the path for which the middleware function is called.
 - **Callback:** It can be a middleware function or a series/array of functions.

EXPRESS APPLICATION FUNCTIONS

- **Example:**

```
const express = require('express');
const app = express();
const PORT = 3000;

app.all('/user', function (req, res, next) {
  console.log('USER API CALLED');
  next();
});

app.listen(PORT, function (err) {
  if (err) console.log(err);
  console.log("Server listening on PORT ", PORT);
});
```

EXPRESS APPLICATION FUNCTIONS

- The **app.delete()** function is utilized to handle **HTTP DELETE** requests for a specified path. It takes the path as a parameter and also accepts callback functions as parameters to handle the request.
- **Syntax:**
`app.delete(path, callback)`
- **Parameters:**
 - **path:** It is the path for which the middleware function is being called.
 - **callback:** It is a middleware function or a series/array of middleware functions.

EXPRESS APPLICATION FUNCTIONS

- **Example:**

```
const express = require('express');
const app = express();
const PORT = 3000;

app.delete('/',
  (req, res) => {
    res.send("DELETE Request Called")
  })

app.listen(PORT,
  function (err) {
    if (err) console.log(err);
    console.log("Server listening on PORT", PORT);
  });
```

EXPRESS REQUEST FUNCTIONS

- The **req.body** property contains key-value pairs of data submitted in the request body. By default, it is undefined and is populated when you use a middleware called body-parsing such as `express.urlencoded()` or `express.json()`.
- **Syntax:**
`req.body`
- **Parameter:** No parameters.
- **Return Value:** Object.

EXPRESS REQUEST FUNCTIONS

- **Example:**

```
const express = require('express');
const app = express();
const PORT = 3000;
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.post('/signup', function (req, res) {
  const data = req.body;

  console.log("Name: ", data.name);
  console.log("Age: ", data.age);
  console.log("Gender: ", data.gender);

  res.send();
});
app.listen(PORT, function (err) {
  if (err) console.log(err);
  console.log("Server listening on PORT", PORT);
});
```

EXPRESS REQUEST FUNCTIONS

- After running the index.js files, make a POST request to **http://localhost:3000/signup** with the following request body:

```
{  
  "name": "John",  
  "age": 24,  
  "gender": "Male"  
}
```

- **Output:** You will see the following output on your console screen:

```
Server listening on PORT 3000  
Name:  John  
Age:   24  
Gender: Male
```

EXPRESS REQUEST FUNCTIONS

- The **req.cookies** property is used when the user is using cookie-parser middleware. This property is an object that contains cookies sent by the request.
- **Syntax:**
`req.cookies`
- **Parameter:** No parameters.
- **Return Value:** Object.

EXPRESS REQUEST FUNCTIONS

- **Example:**

```
const cookieParser = require('cookie-parser');
const express = require('express');
const app = express();
const PORT = 3000;
app.use(cookieParser());
app.get('/user', function (req, res) {
  req.cookies.name = 'John';
  req.cookies.age = 24;

  console.log(req.cookies);
  res.send();
});
app.listen(PORT, function (err) {
  if (err) console.log(err);
  console.log("Server listening on PORT", PORT);
});
```

EXPRESS REQUEST FUNCTIONS

- After running the index.js files, open your browser and make a GET request to **http://localhost:3000/user**, now you can see the following output on your console:

```
Server listening on PORT 3000
```

```
[Object: null prototype] { name: 'John', age: 24 }
```

EXPRESS RESPONSE FUNCTIONS

- The **res.end()** function concludes the response process and is derived from the `HTTP.ServerResponse`'s **response.end()** method in the Node core. It is employed to promptly conclude the response without including any data.
- **Syntax:**

```
res.end([data] [, encoding])
```
- **Parameters:** The default encoding is **'utf8'** and the data parameter is the data with which the user wants to end the response.
- **Return Value:** It returns an Object.

EXPRESS RESPONSE FUNCTIONS

- **Example:**

```
const express = require('express');
const app = express();
const PORT = 3000;

// Without middleware
app.get('/',
  function (req, res) {
    res.end();
  });

app.listen(PORT,
  function (err) {
    if (err) console.log(err);
    console.log("Server listening on PORT", PORT);
  });
```

EXPRESS RESPONSE FUNCTIONS

- The `res.cookie()` function is used to set a cookie in the client's browser. It allows you to assign a cookie by providing a name and a value. The value can be a simple string or an object, which will be automatically converted to JSON.
- **Syntax:**

```
res.cookie(name, value [, options])
```

 - **name:** The name of the cookie.
 - **value:** The value of the cookie. It can be a string or an object that is converted to JSON.
 - **options (Optional):** An object that can contain properties like `expires`, `domain`, `path`, `secure`, etc. These help control the behavior of the cookie.
- **Return Value:** It returns an `Object`.

EXPRESS RESPONSE FUNCTIONS

- **Example:**

```
const express = require('express');
const app = express();
const PORT = 3000;

app.get('/', function (req, res) {
  // Setting a simple cookie: 'name' = 'simplecookie'
  res.cookie('name', 'simplecookie');
  res.send("Cookie Set");
});

app.listen(PORT, function (err) {
  if (err) console.log(err);
  console.log("Server listening on PORT", PORT);
});
```

EXPRESS RESPONSE FUNCTIONS

- The **res.clearCookie()** function is used to clear the cookie specified by name. This function is called for clearing the cookies which have already been set. For example, if a *user* cookie is set, then it can be cleared using this function.
- **Syntax:**

```
res.clearCookie(name, [ options ])
```
- **Parameters:**
 - **Name:** It is the name of the cookie which is to be cleared. Like in the following example, we have cleared the *title* cookie.
 - **Options:** It is an object that can have various properties like domain, encode, path, secure, etc.

EXPRESS RESPONSE FUNCTIONS

- **Example:**

```
const express = require('express');
const app = express();
const PORT = 3000;
app.get('/', function (req, res) {

    // Setting cookie (key-value)
    res.cookie('title', 'thisisacookie');
    // Clearing the cookie
    res.clearCookie('title');
    console.log("Cookie cleared");
});
app.listen(PORT, function (err) {
    if (err) console.log(err);
    console.log("Server listening on PORT", PORT);
});
```


EXPRESS ROUTER FUNCTIONS

- The **router.METHOD()** method provides the routing functionality in Express, where METHOD is one of the HTTP methods, such as GET, PUT, POST, and so on, in lowercase.
- **Syntax:**
`router.METHOD(path, [callback, ...] callback)`
- **Parameter:** The path parameter specifies the path on the URL and callback is the function that is executed when this path is called.
- **Return Value:** Since, it provides the routing functionality, so it can return responses.

EXPRESS ROUTER FUNCTIONS

```
const express = require('express');
const app = express();
const router = express.Router();
const PORT = 3000;
// Single route
router.get('/user', function (req, res, next) {
  console.log("GET request called");
  res.end();
});
app.use(router);
app.listen(PORT, function (err) {
  if (err) console.log(err);
  console.log("Server listening on PORT", PORT);
});
```

EXPRESS ROUTER FUNCTIONS

- The **router.use()** function uses the specified middleware function or functions. It basically mounts middleware for the routes which are being served by the specific router.

- **Syntax:**

```
router.use( path, function )
```

- **Parameters:**

- **Path:** It is the path to this middleware, if we can have */user*, now this middleware is called for all API's having */user* of this router.
- **function:** This function is passed as a callback, it is called when the specified path is called in this router.

EXPRESS ROUTER FUNCTIONS

- **Example:**

```
const express = require('express');
const app = express();
const router = express.Router();
const PORT = 3000;
// All requests to this router will
// first hit this middleware
router.use(function (req, res, next) {
  console.log("Middleware Called");
  next();
})
// Always invoked
router.use(function (req, res, next) {
  res.send("Greetings!");
})
app.use('/user', router);
app.listen(PORT, function (err) {
  if (err) console.log(err);
  console.log("Server listening on PORT", PORT);
});
```